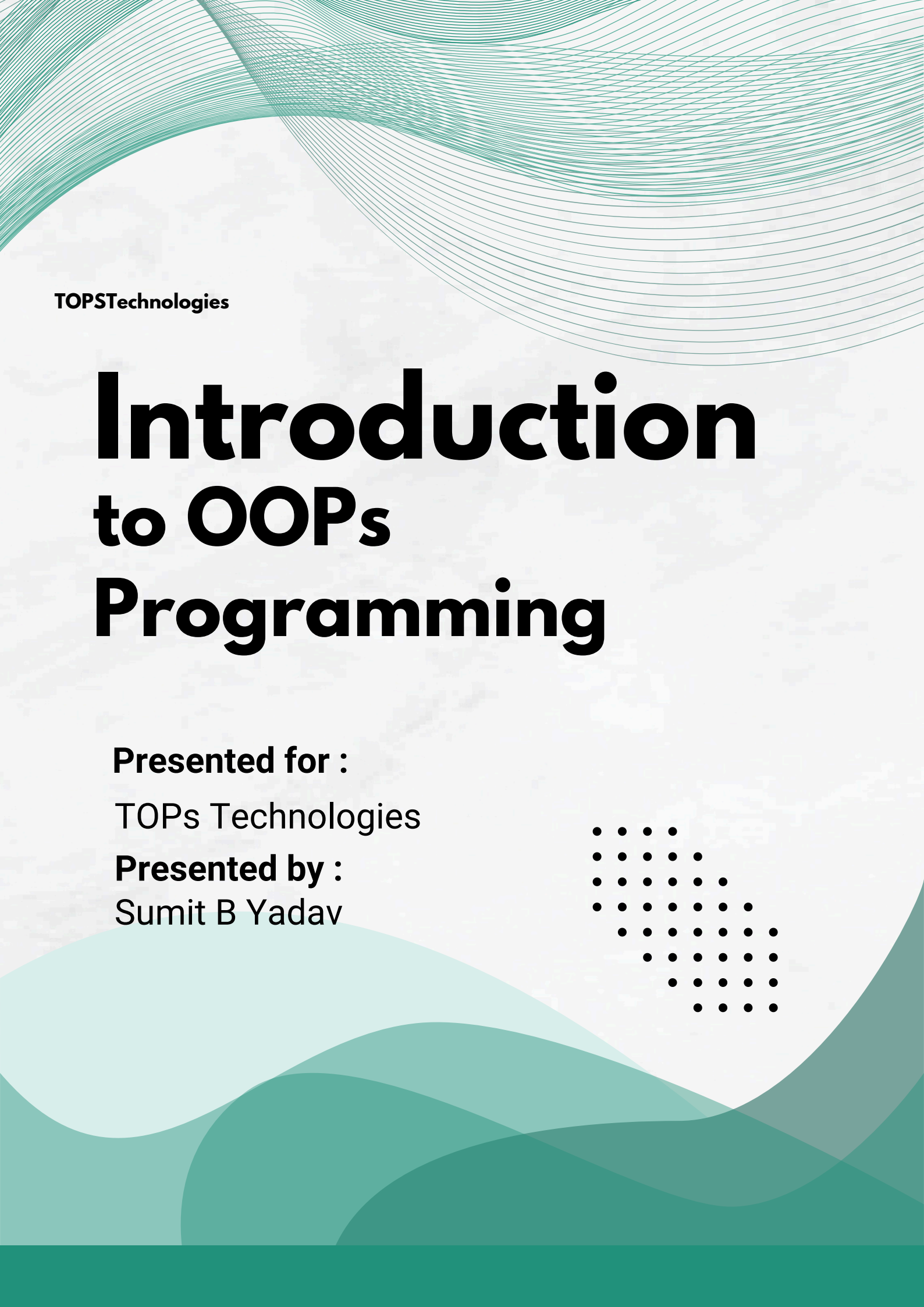




TOPSTechnologies

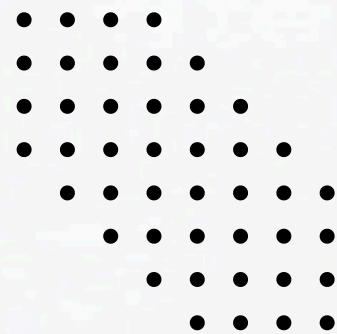
Introduction to OOPs Programming

Presented for :

TOPs Technologies

Presented by :

Sumit B Yadav



Basic Structure of a C++ Program

Que 1

In C++, there are several built-in data types that represent different kinds of data. These data types can be broadly categorized into the following groups:

1. Basic Data Types

These are the most fundamental data types in C++.

- **Integer Types:** Used to represent whole numbers, both positive and negative.

int: Typically used for integers.

long: A larger integer, typically 4 bytes or more.

Floating Point Types: Used to represent numbers with decimal points.

- **float:** Single-precision floating-point (typically 4 bytes).
- **double:** Double-precision floating-point (typically 8 bytes).
- **long double:** Extended precision floating-point.
- **char:** Typically used for single characters, typically 1 byte
- **Arrays:** A collection of elements of the same type.

Basic Structure of a C++ Program

Que2

1. Implicit Type Conversion (Automatic Type Conversion)

Implicit type conversion is also known as automatic type conversion or type coercion. It happens automatically when the compiler converts one data type to another without any explicit request from the programmer.

This type of conversion is performed when the conversion is safe and doesn't lead to loss of data or precision.

Characteristics:

- *Performed by the compiler:* Implicit type conversion is automatically done by the compiler when necessary.
- *No user intervention required:* The programmer doesn't have to specify anything.
- *Safe conversions:* It typically happens between compatible types, where the conversion doesn't result in a loss of information (e.g., from int to float).
- *Widening conversions:* It usually occurs when converting from a smaller data type to a larger one, such as from int to double or char to int.

2. Explicit Type Conversion (Manual Type Conversion or Casting)

Explicit type conversion, also called type casting, requires the programmer to manually specify how the conversion should be done.

This is often used when the compiler cannot automatically convert types (e.g., from a larger type to a smaller type, where data might be lost), or when the programmer wants to override the default behavior.

Basic Structure of a C++ Program

Characteristics:

- *Performed by the programmer: The programmer explicitly specifies the conversion.*
- *Can involve potential data loss: In cases like casting from double to int, precision might be lost.*
- *Narrowing conversions: It often involves converting a larger data type to a smaller one, like from double to int.*
- *C++ provides casting operators: C++ supports several ways to perform explicit type conversion, such as C-style casting, static_cast, dynamic_cast, const_cast, and reinterpret_cast.*

Que 3

Different Types of Operators in C++

C++ provides a rich set of operators that allow manipulation of data and variables. These operators can be broadly classified into several categories:

1.1 Arithmetic Operators

These operators are used to perform mathematical operations on numeric data types.

- *+ (Addition): Adds two operands.*
- *- (Subtraction): Subtracts the second operand from the first.*
- ** (Multiplication): Multiplies two operands.*
- */ (Division): Divides the first operand by the second.*
- *% (Modulus): Returns the remainder of the division.*

Basic Structure of a C++ Program

Relational Operators

Relational operators are used to compare two values or expressions.

- `==` (Equal to): Returns true if the operands are equal.
- `!=` (Not equal to): Returns true if the operands are not equal.
- `>` (Greater than): Returns true if the left operand is greater than the right.
- `<` (Less than): Returns true if the left operand is less than the right.
- `>=` (Greater than or equal to): Returns true if the left operand is greater than or equal to the right.
- `<=` (Less than or equal to): Returns true if the left operand is less than or equal to the right.

Logical Operators

- Logical operators are used to combine multiple conditions (i.e., boolean expressions).
- `&&` (Logical AND): Returns true if both operands are true.
- `||` (Logical OR): Returns true if at least one operand is true.
- `!` (Logical NOT): Reverses the logical state of its operand (i.e., converts true to false and vice versa).

Assignment Operators

Assignment operators are used to assign values to variables.

- `=`: Assigns the right operand to the left operand.
- `+=`: Adds the right operand to the left operand and assigns the result to the left.
- `-=`: Subtracts the right operand from the left operand and assigns the result to the left.
- `*=`: Multiplies the left operand by the right operand and assigns the result to the left.

Basic Structure of a C++ Program

- */=: Divides the left operand by the right operand and assigns the result to the left.*
- *%=: Takes the modulus of the left operand by the right operand and assigns the result to the left.*

Increment and Decrement Operators

These operators are used to increment or decrement a variable by one.

- *++ (Increment): Increases the value of the operand by 1.*
- *-- (Decrement): Decreases the value of the operand by 1.*

Bitwise Operators

Bitwise operators operate on the binary representations of numbers.

- *& (AND): Performs a bitwise AND operation.*
- *| (OR): Performs a bitwise OR operation.*
- *^ (XOR): Performs a bitwise XOR operation.*
- *~ (NOT): Performs a bitwise NOT operation.*
- *<< (Left Shift): Shifts the bits of the left operand to the left by the number of positions specified by the right operand.*
- *>> (Right Shift): Shifts the bits of the left operand to the right by the number of positions specified by the right operand.*

Basic Structure of a C++ Program

Ternary Operator

The ternary operator is a shorthand for an if-else statement.

- *? :: condition ? expr1 : expr2 – if the condition is true, expr1 is evaluated; otherwise, expr2 is evaluated.*

Que 3

Break Statement

The break statement is used to exit a loop prematurely. When a break is encountered inside a loop (whether for, while, or do-while), the loop is terminated immediately, and the program execution continues with the statement after the loop.

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; ++i) {
        if (i == 6) {
            break; // Exit the loop when i reaches 6
        }
        cout << i << " ";
    }
    cout << "\nLoop ended." << endl;
    return 0;
}
```

Basic Structure of a C++ Program

Continue Statement

The continue statement is used to skip the current iteration of a loop and continue with the next iteration. When the continue statement is encountered, the rest of the code in the loop for the current iteration is skipped, and the loop proceeds to the next iteration (if any).

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; ++i) {
        if (i == 5) {
            continue; // Skip the iteration when i is 5
        }
        cout << i << " ";
    }
    return 0;
}
```